
Redes Neurais e Aprendizagem Profunda

— Redes Neurais Convolucionais —

Sumário

Córtex visual

Camadas convolucionais

Camadas de pooling

Arquitetura típica

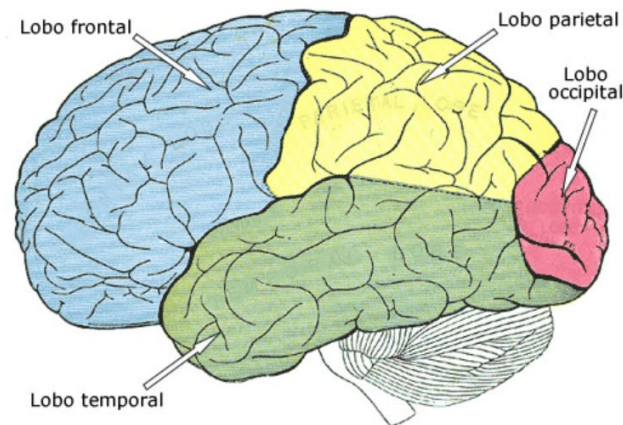
Tarefas aparentemente triviais?

- 1996
 - Deep Blue venceu Garry Kasparov
- Após 2012
 - Detetar um cão numa imagem
 - Reconhecer palavras faladas
- Percepção acontece fora do domínio da consciência
 - módulos sensoriais especializados de visão, audição e outros
 - percepção não é trivial
- Redes neuronais convolucionais
 - emergiram do estudo do córtex visual

Córtex visual

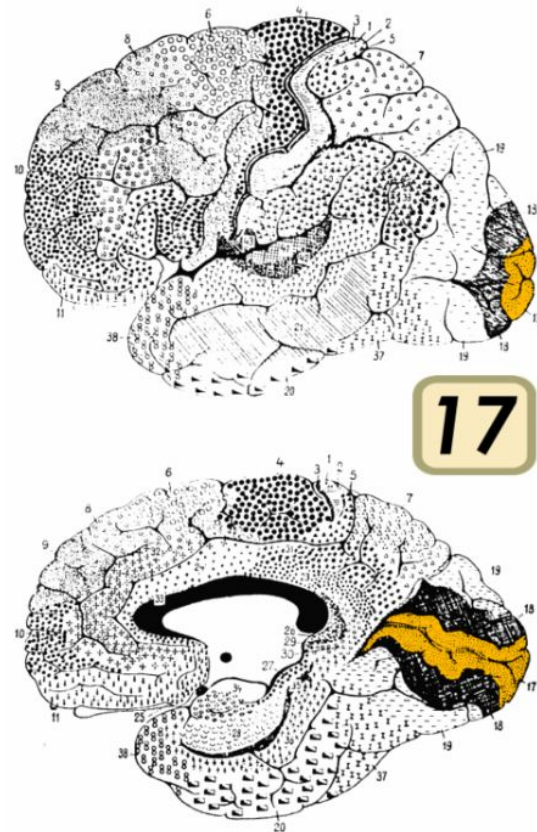
Córtex cerebral

- **Córtex cerebral**
 - camada mais externa do cérebro
 - rico em neurónios
 - local de processamento mais sofisticado
- **Córtex humano**
 - espessura: 2-4 mm
 - área: 0.22 m²
 - 14-16 biliões de neurónios
 - dividido em 4 lobos principais: frontal, parietal, occipital e temporal
- **Papel central em funções complexas**
 - memória, atenção, consciência, linguagem, percepção



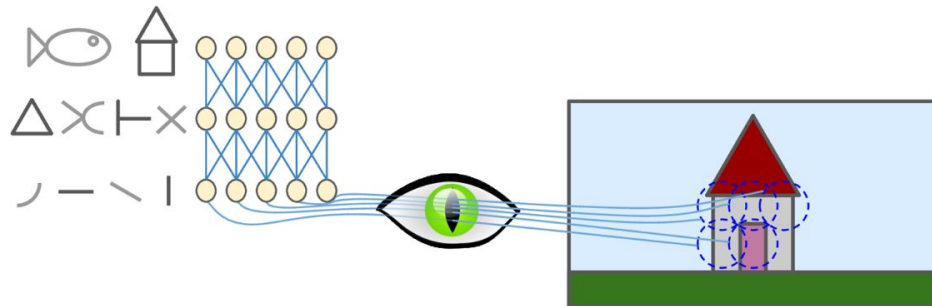
Córtex visual

- Processa a informação visual
- Região
 - Lobo occipital
- Composto por várias áreas
 - principal: V1 (área Brodmann 17)
 - outras: V2 - V5



Arquitetura córtex visual

- 1958, 1959 (Hubel e Wiesel)
 - campo local de receção pequeno
 - funções especializadas



- Cada neurónio
 - reage a estímulos visuais localizados numa região limitada do campo visual
 - pode sobrepor-se ao campo visual de outros neurónios
 - juntos preenchem todo o campo visual
 - reage a imagens com diferentes padrões
 - linhas com diferentes orientações
 - pode ter campos de receção mais amplos que reagem a padrões que combinam padrões mais simples

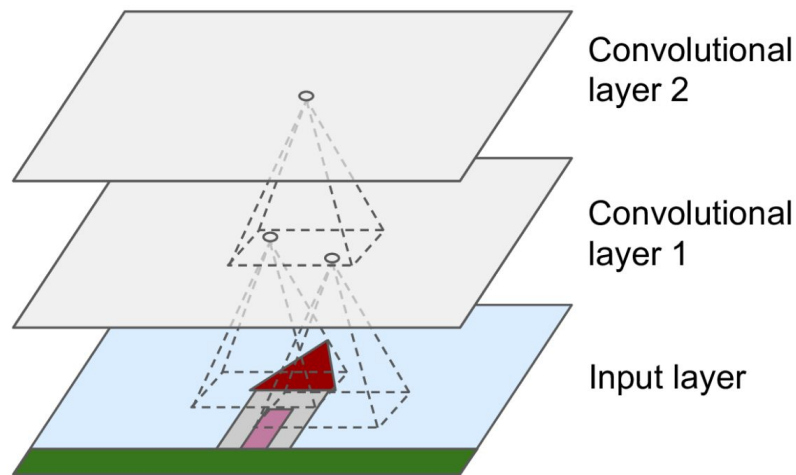
Evolução

- 1980, Fukushima
 - Neocognitron
 - Evoluiu gradualmente para rede neuronal convolucional
- 1998, LeCun
 - arquitetura LeNet
 - muito utilizada para reconhecer algarismos manuscritos em cheques bancários
- LeNet
 - camadas densas (fully connected)
 - função de ativação sigmoide
 - camadas convolucionais
 - camadas pooling

Camadas convolucionais

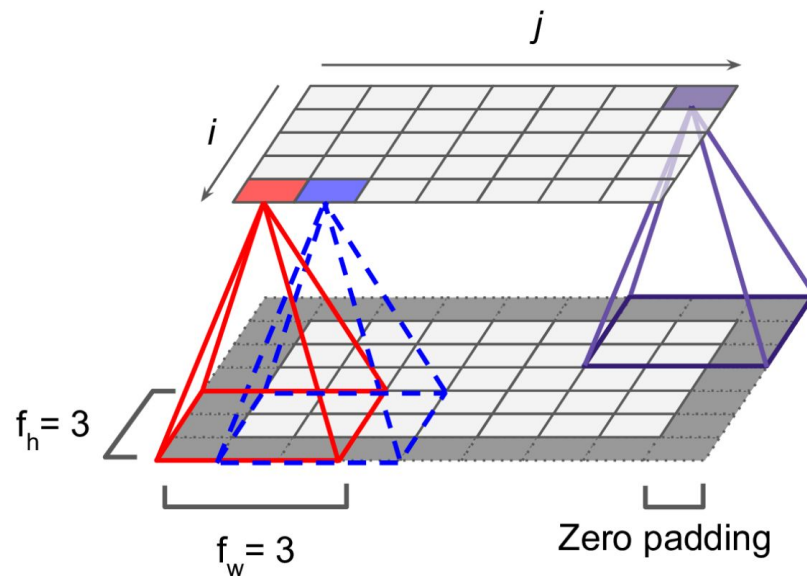
Camadas convolucionais

- 1ª camada
 - cada neurónio ligado apenas a alguns pixels (campo de receção)
 - concentra-se em características de baixo nível
- 2ª camada
 - cada neurónio ligado a neurónios de um pequeno rectângulo da 1ª camada
 - constrói características de nível mais alto a partir das características da 1ª camada
- Esta estrutura hierárquica é comum em imagens reais!



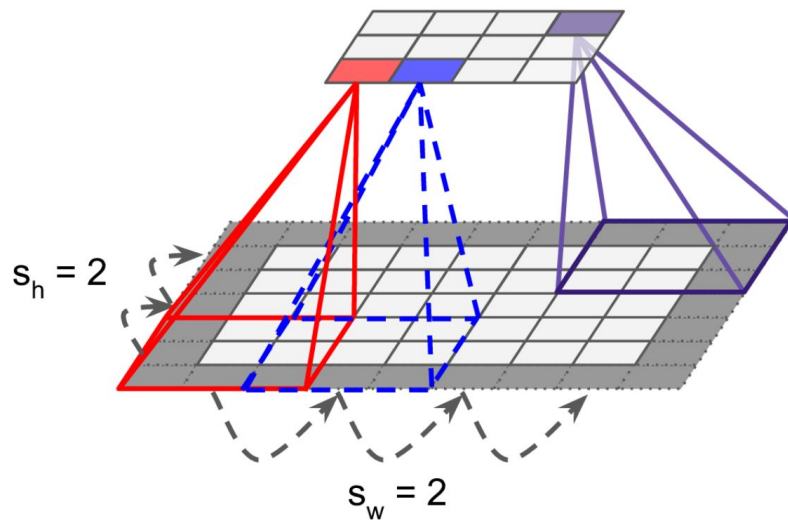
Campo de recepção

- neurónio(i,j) ligado aos neurónios
 - linhas: i a $i+f_h-1$
 - colunas: j a $j+h_w-1$
 - f_h : altura do campo de receção
 - f_w : largura do campo de receção
- zero padding
 - adição de zeros à volta das entradas
 - mantém camadas com o mesmo tamanho
- Exemplo
 - camada de entrada: 5×7 + zero padding
 - campo de receção: 3×3
 - camada seguinte: 5×7



Stride

- Deslocação do um campo de receção
 - reduz do tamanho da camada seguinte
 - reduz a complexidade do modelo
- neurónio(i,j) ligado aos neurónios
 - linhas $i*s_h$ a $i*s_h+f_h-1$
 - colunas $j*s_w$ a $j*s_w+h_w-1$
 - s_h : stride vertical
 - s_w : stride horizontal
- Exemplo
 - camada de entrada: 5×7 + zero padding
 - campo de receção: 3×3
 - stride: 2×2
 - camada seguinte: 3×4

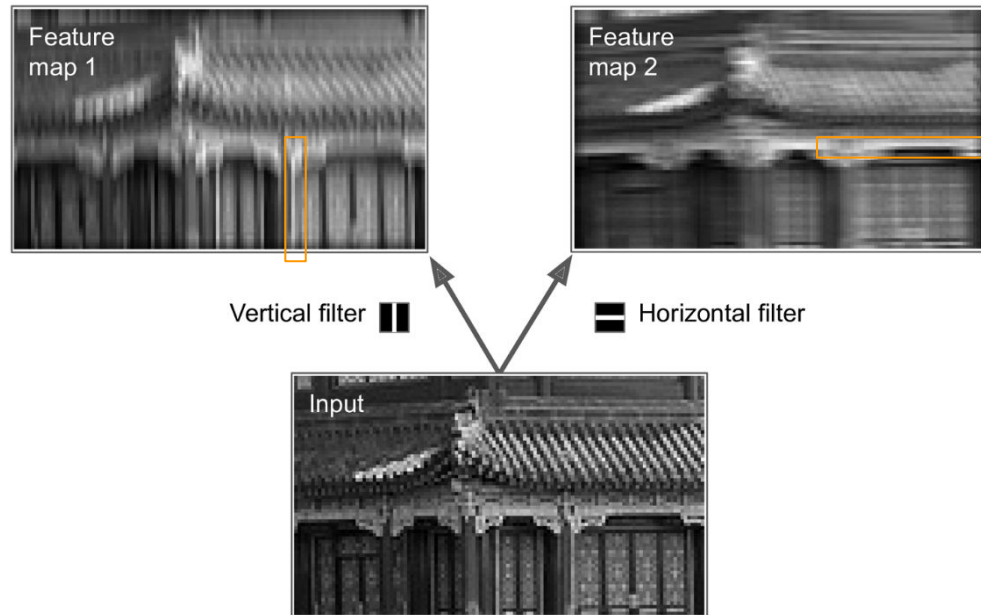


Filtro e mapas de características

- Filtro
 - Conjunto de pesos
 - Kernel convolucional
 - Pode ser representado por uma imagem do tamanho do campo de recepção
- Mapa de características
 - Resultado da aplicação de um filtro a toda a imagem
 - Realça as áreas da imagem que ativam o mais o filtro

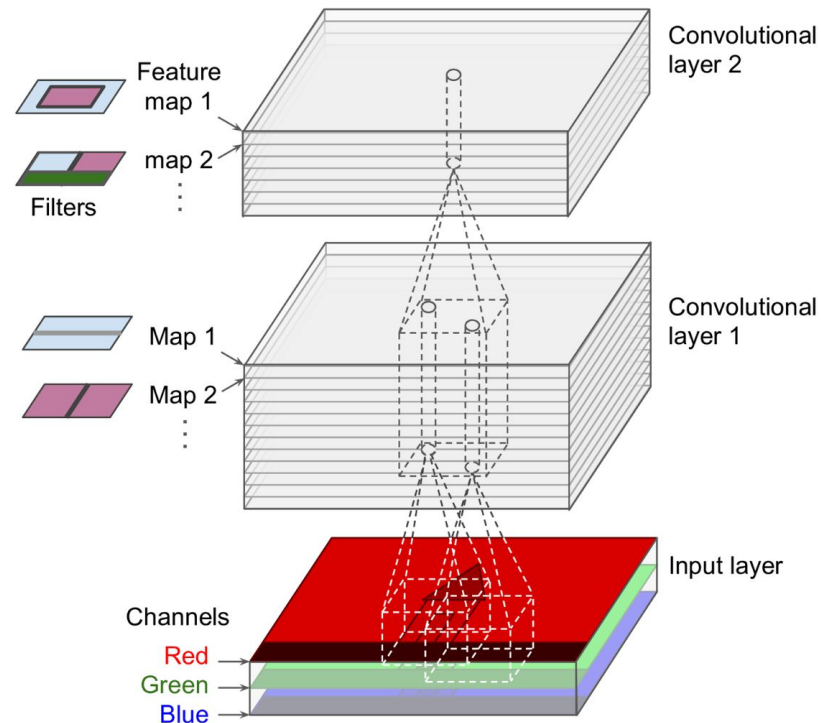
Exemplo

- filtro vertical
 - matriz 7x7 c/ 0s exceto coluna central
 - neurónios ignoram tudo exceto a linha central vertical
- filtro horizontal
 - matriz 7x7 c/ 0s exceto linha central
 - neurónios ignoram tudo exceto a linha central horizontal



Camada convolucional

- Composta por múltiplos filtros (treináveis)
- Gera um mapa de características por filtro
- Representada por uma camada 3D
 - um neurónio por pixel em cada mapa
 - todos os neurónios de um mapa partilham os mesmos pesos
 - o campo de receção estende-se por todos os mapas da camada anterior
- Aplica múltiplos filtros às suas entradas permitindo detetar múltiplas características em qualquer posição



Canais

- Sub-camadas da camada de entrada
- Tons de cinzento
 - 1 canal
- Imagem RGB
 - 3 canais: red, green, blue
- Imagem multi-espectral
 - dezenas
- Imagem hiper-espectral
 - centenas

Saída do neurónio

- neurónio(i,j,k,l) está ligado aos neurónios
 - linhas $i \cdot s_h$ a $i \cdot s_h + f_h - 1$
 - colunas $j \cdot s_w$ a $j \cdot s_w + h_w - 1$
 - da camada l-1
 - em todos os mapas de características
- saída
 - calcula a soma pesada de todas as entradas (mais o termo bias)
 - i: linha j: coluna
 - l: camada k: mapa de características
 - f_h : número de mapas da camada anterior (l-1)
 - $x_{i',j',k'}$: saída do neurónio da camada l-1, linha i', coluna j', mapa k'
 - $w_{u,v,k',k}$: peso entre qualquer neurónio do mapa k, camada l, entrada da linha u, coluna v (relativa ao seu campo de receção) e o mapa k'

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_n-1} x_{i',j',k'} \cdot w_{u,v,k',k} \quad \text{with} \quad \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

Implementação Tensorflow

- imagem
 - tensor 3D de formato [altura, largura, canais]
- mini-batch
 - tensor 4D de formato [tamanho mini-batch, altura, largura, canais]
- camada convolucional
 - pesos: tensor 4D de formato $[f_h, f_w, f_{n'}, f_n]$
 - bias: tensor 1D de formato $[f_n]$
 - n: nº de filtros

Exemplo

```
from sklearn.datasets import load_sample_image
```

```
# Load sample images
```

```
china = load_sample_image("china.jpg") / 255
```

```
flower = load_sample_image("flower.jpg") / 255
```

```
images = np.array([china, flower])
```

```
batch_size, height, width, channels = images.shape
```

```
# Create 2 filters
```

```
filters = np.zeros(shape=(7, 7, channels, 2), dtype=np.float32)
```

```
filters[:, 3, :, 0] = 1 # vertical line
```

```
filters[3, :, :, 1] = 1 # horizontal line
```

```
outputs = tf.nn.conv2d(images, filters, strides=1, padding="same")
```

```
plt.imshow(outputs[0, :, :, 1], cmap="gray") # plot 1st image's 2nd feature
```

```
map
```

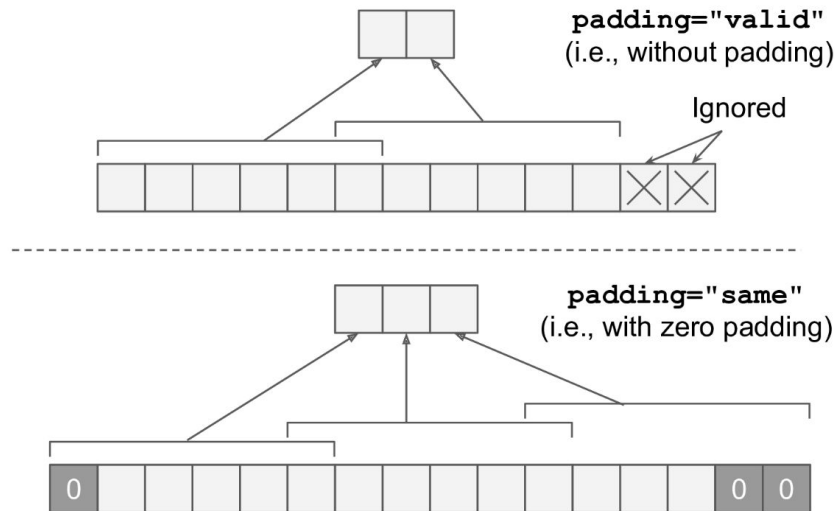
```
plt.show()
```

instalar pacote `Pillow` para usar `load_sample_image()`

- representação da intensidade como um valor entre 0 e 1
- criação de 2 filtros 7x7
 - linha vertical branca no meio
 - linha horizontal branca no meio
- aplicação dos filtros a ambas as imagens
 - padding same
 - stride 1
- mostrar o 2º mapa da 1ª imagem

Padding

- VALID
 - cada campo de receção está dentro dos limites válidos da entrada
 - não usa padding zero
 - pode ignorar algumas linhas e colunas em baixo e à direita da imagem
- SAME
 - usa padding zero se necessário
 - tamanho saída
 - nº neurónios entrada divididos pelo stride (arredondado cima)
 - zeros adicionados à volta das entradas com espaçamento uniforme



Camada convolucional

- Os filtros são variáveis treináveis
 - para a rede aprender os quais os melhores
- Hiper-parâmetros
 - n° filtros
 - tamanho filtros
 - strides, padding
- Exemplo
 - 32 filtros de tamanho 3x3, stride=1, zero padding
 - função de ativação ReLU

```
conv = keras.layers.Conv2D(filters=32, kernel_size=3, strides=1,  
                             padding="same", activation="relu")
```

Requisitos de memória

- Requer grande quantidade de memória
- Durante treino
 - todos os cálculos do passo forward têm de ser mantidos para o passo backward
 - total de memória requerida por todas as camadas
- Durante previsão
 - a memória ocupada por uma camada pode ser libertada assim que a próxima camada for calculada

Exemplo

- imagem RGB 150x100

Camada densa

- nº parâmetros
 - 150*100 neurónios
 - cada neurónio ligado
 - 150x100x3 entradas
 - total=1675 milhões

Camada convolucional

- Características
 - filtros 5x5
 - 200 mapas de tamanho 150x100
 - stride=1, padding zero
- nº parâmetros
 - $(5*5*3+1)*200=15200$
- nº calculos
 - cada mapa
 - 150x100 neurónios
 - cada neurónio
 - calcula soma pesada de 5*5*3 entradas
 - total
 - 225 milhões de multiplicações vírgula flutuante
- memória
 - mapa representado por floats 32 bits
 - $200*150*100*32 = 12$ MB por imagem
 - batch=100 img -> 1.2 GB RAM

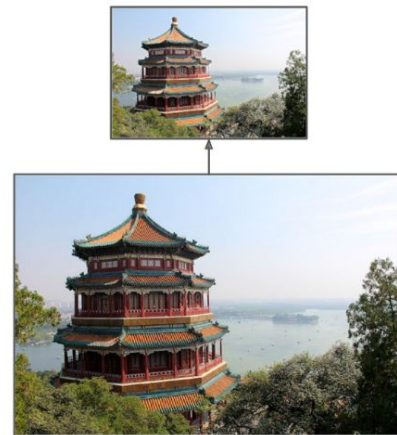
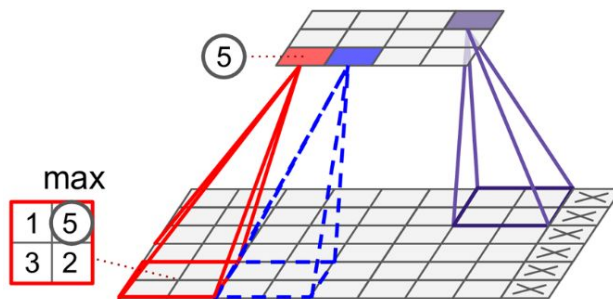
Camadas de pooling

Objetivo

- Reduzir
 - peso computacional
 - utilização de memória
 - número de parâmetros
- Como
 - sub-amostragem da entrada
- Neurónio
 - ligado a um nº limitado de neurónios da camada anterior
 - Campo de receção: definição do tamanho, stride, padding
 - não tem pesos
 - agrega as entradas: max, média

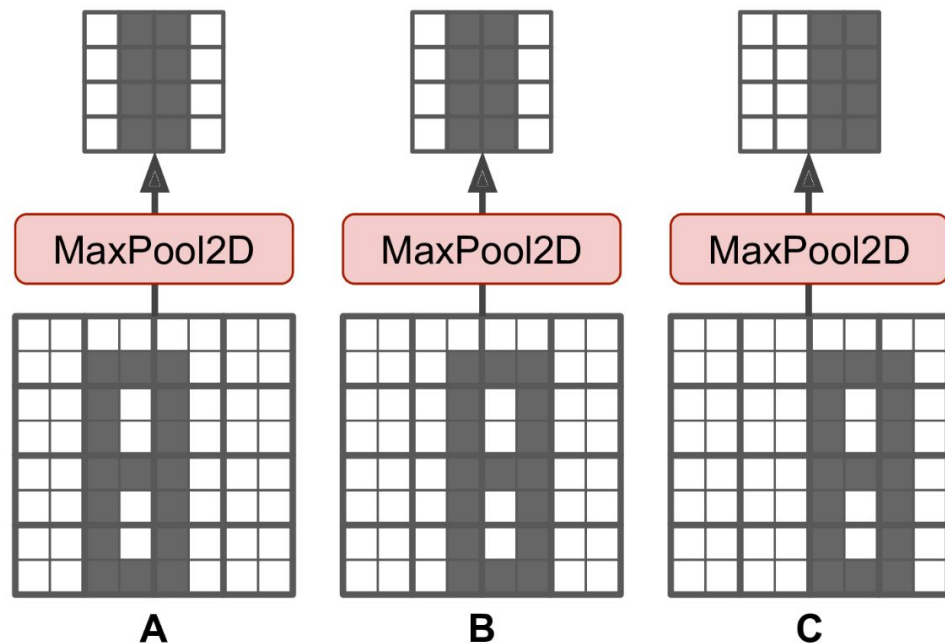
Exemplo

- Max-pooling
 - Tipo de camada mais comum
- Características
 - tamanho: 2x2
 - stride: 2
 - padding: --
- Imagem de saída
 - metade do tamanho da entrada (stride=2), arredondado para baixo (no padding)



Característica

- Introduce **invariância** em translações pequenas
- Exemplo
 - pixels escuros: valor maior
 - max-pooling
 - kernel: 2x2
 - stride 2
 - $\text{max-pooling}(B) = \text{max-pooling}(A)$
 - $\text{max-pooling}(C) \sim \text{max-pooling}(A)$ (75% invariante)
- alguma invariância a rotação e escala
- Útil para problemas onde a previsão não depende destes detalhes



Desvantagens

- É destrutiva
 - filtro=2x2, stride=2
 - saída 2x mais pequena em ambas as direções
 - 75% dos valores são destruídos
- Em algumas aplicações a invariância não é desejável

MaxPool vs AveragePool

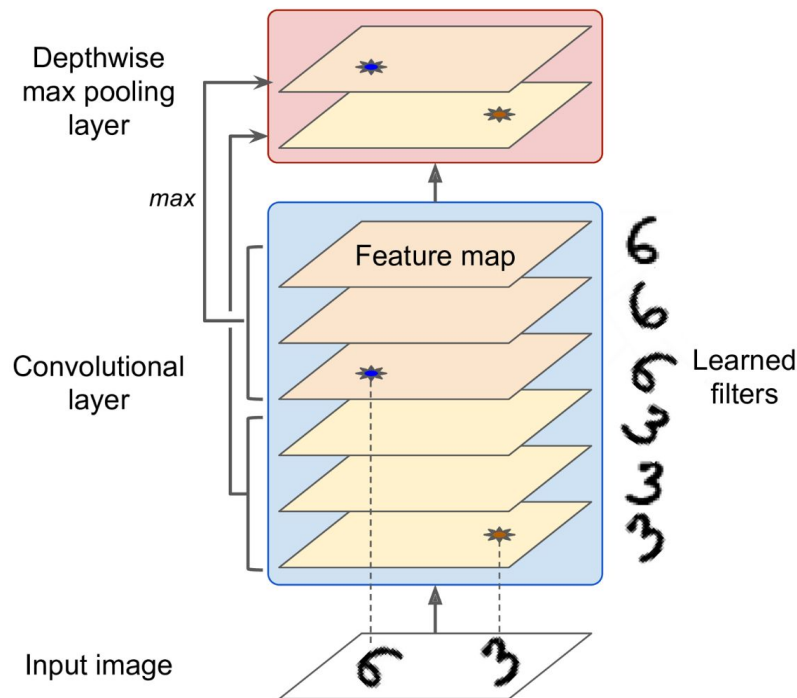
- AveragePool
 - foi popular
 - geralmente perde-se menos informação
- MaxPool
 - normalmente têm melhor desempenho
 - mantém as características mais fortes
 - camadas seguintes recebem um sinal mais limpo
 - maior invariância translacional
 - menor computação

Implementação Tensorflow

- `MaxPoolLayer()`
- `AveragePoolLayer()`
- por omissão `stride=tamanho filtro`

Pooling em profundidade

- Normalmente o pooling é aplicado espacialmente
 - aplicado de forma independente sobre cada mapa de características
- É possível aplicar pooling sobre os mapas
 - permite à rede aprender a ser invariante a várias características
 - rotação, brilho, cor, espessura, etc
- Exemplo
 - filtros detetam diferentes rotações do mesmo padrão
 - assegura a mesma saída independentemente da rotação



Pooling média global

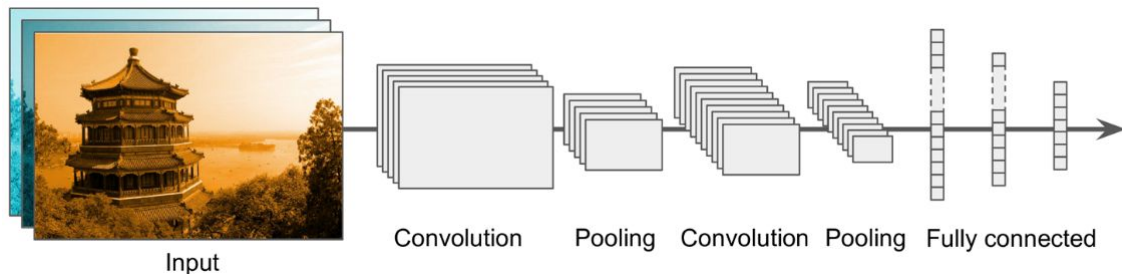
- calcula a média de cada mapa de características
 - equivalente a AveragePooling com kernel=dimensão espacial da entrada
- gera um valor por mapa
- keras
 - `GlobalAvgPool2D()`

Arquitetura típica

Arquitetura típica

- Empilha
 - Algumas camadas convolucionais (seguidas de uma camada ReLU)
 - Uma camada de pooling
 - Mais camadas convolucionais (+ReLU)
 - Outra camada de pooling
 - ...
 - algumas camadas densas (+ReLU)
 - camada de saída que gera a previsão (softmax)

- A imagem fica cada vez mais pequena, mas mais profunda!



Tamanho dos filtros

- Empilhar 2 camadas de filtros 3x3 vs. 1 camada 5x5
 - mais eficiente
 - menos parâmetros, menos cálculos
 - melhor desempenho
- Exceção
 - primeira camada
 - tipicamente tem um kernel grande (5x5), com stride 2 ou mais
 - reduz a dimensão da imagem sem perder muita informação

Exemplo

```
model = keras.models.Sequential([
    keras.layers.Conv2D(64, 7, activation="relu", padding="same",
                        input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(128, 3, activation="relu", padding="same"),
    keras.layers.Conv2D(128, 3, activation="relu", padding="same"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(256, 3, activation="relu", padding="same"),
    keras.layers.Conv2D(256, 3, activation="relu", padding="same"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, activation="softmax")
])
```

- Tamanho da imagem?
- N° de canais?
- Arquitetura
 - 1ª camada
 - tipo
 - n° de filtros
 - tamanho filtro
 - stride
 - padding
 - 2ª camada
 - tipo
 - ...
 - outras camadas

Desempenho sobre Fashion MNIST

- Arquitetura densa
 - exatidão: 88%
- Arquitetura CNN
 - exatidão: 92%

Variantes

- Ao longo dos anos surgiram diversas variantes
 - LeNet-5
 - AlexNet
 - GoogLeNet
 - VGGNet
 - ResNet
 - Xception
 - SNet
 - ...
- Medida do progresso
 - Taxa de erro em competições
- ILSVRC
 - ImageNet Large Scale Visual Recognition Challenge
 - <https://image-net.org/challenges/LSVRC/index.php>
 - características
 - tamanho da imagem: 256x256
 - classes: 1000
 - erro top-5 em 6 anos
 - $> 26\% \rightarrow < 2.3\%$

Bibliografia

Capítulo 14

*Hands-On Machine Learning with
Scikit-Learn, Keras & TensorFlow*
